Relazione - Versione 5: Embedding e Finestra di contesto

Introduzione

La Versione 5 estende il modello neurale di linguaggio a caratteri in TensorFlow della Versione 4 introducendo embedding appresi e una finestra di contesto fissa.
Corpus, vocabolario, split di training e validation, mini-batch mescolati, obiettivo cross-entropy, differenziazione automatica e campionamento autoregressivo restano invariati. L'input one-hot di un solo carattere viene sostituito da quattro identificatori di carattere, i cui vettori di embedding appresi vengono combinati per prevedere il carattere successivo.

```
Caratteri
↓
Identificatori interi
↓
Contesto di 4 caratteri
↓
Embedding addestrabili
↓
Vettore di contesto appiattito
↓
Pesi di output
↓
Testo generato
```

Obiettivo della Versione 5

Il modello deve essere in grado di:

- riutilizzare lo stesso corpus di 440 caratteri e il vocabolario di 27 caratteri della Versione 4;
- sostituire gli input one-hot di un singolo carattere con contesti fissi di quattro identificatori di carattere;
- associare ogni identificatore di carattere a un vettore di embedding addestrabile a 8 dimensioni;
- concatenare quattro embedding in una rappresentazione del contesto di 32 valori;
- apprendere sia la matrice di embedding sia i pesi di output con tf.GradientTape;
- mantenere training mini-batch, monitoraggio della validation e generazione autoregressiva riproducibile;
- verificare costruzione del contesto, forme del modello, gradienti, aggiornamenti dei parametri, retraining deterministico e generazione.

Architettura

Il flusso completo della Versione 5 è:

```
Corpus: 440 caratteri
↓
Vocabolario: 27 caratteri
↓
436 esempi contesto-target da 4 caratteri
↓
Tensore input intero: (436, 4)
↓
Matrice di embedding: (27, 8)
↓
Contesto appiattito: 32 valori
↓
Matrice pesi di output: (32, 27)
↓
Split esempi 80 / 20 riproducibile
↓
Aggiornamenti mini-batch con GradientTape
↓
Generatore autoregressivo a finestra scorrevole
```

Il contesto contiene ora quattro caratteri precedenti. La Versione 5 supera quindi il bigramma neurale a un carattere usato nella Versione 4, mantenendo il modello volutamente piccolo e ispezionabile.

Finestre di contesto, split e mini-batch

Le finestre scorrevoli creano un esempio input-target per ogni posizione in cui sono disponibili quattro caratteri precedenti. Lo stesso generatore NumPy locale con seed 42 mescola i 436 indici degli esempi e le operazioni gather di TensorFlow selezionano i tensori di training e validation.

```
esempi training = 348
esempi validation = 88
dimensione batch = 32
aggiornamenti per epoca = 11
```

Solo gli esempi di training vengono mescolati e usati per aggiornare i parametri. Gli esempi di validation misurano la loss senza modificare il modello. Poiché le finestre di contesto vicine si sovrappongono, lo split a livello di esempio mantiene la stessa limitazione intenzionale legata al piccolo dataset delle versioni precedenti.

Embedding addestrabili e pesi di output

Ogni identificatore di carattere seleziona una riga da una matrice di embedding addestrabile. I quattro vettori selezionati vengono concatenati in una rappresentazione a dimensione fissa, che viene moltiplicata per una matrice di output addestrabile per produrre i logits del carattere successivo.

```
Forma tensore input: (436, 4)
Forma matrice embedding: (27, 8)
Forma contesto embedded: (1, 4, 8)
Forma contesto appiattito: (1, 32)
Forma matrice pesi output: (32, 27)
Parametri addestrabili: 1080
```

La matrice di embedding contiene 216 valori e la matrice di output ne contiene 864. Entrambe sono memorizzate come variabili TensorFlow e usano calcoli float32.

Loss stabile in TensorFlow

Gli identificatori del contesto vengono mappati negli embedding, rimodellati in un unico vettore di contesto e moltiplicati per i pesi di output. La sparse softmax cross-entropy misura l'errore di previsione direttamente dai logits e dagli ID target interi.

```
embeddings = tf.gather(embedding_matrix, inputs)
context_vectors = tf.reshape(
embeddings, (batch_size, 32)
)
logits = tf.matmul(context_vectors, output_weights)
example_losses = tf.nn.sparse_softmax_cross_entropy_with_logits(
labels=targets, logits=logits
)
loss = tf.reduce_mean(example_losses)
```

Le loss iniziali di training e validation sono entrambe circa 3.2958, un valore coerente con una previsione quasi uniforme sul vocabolario di 27 caratteri.

Esecuzione controllata e riproducibilità

Il notebook mantiene l'esecuzione su CPU e sopprime i log TensorFlow di basso livello perché il modello è piccolo e non richiede CUDA.

Seed espliciti controllano l'inizializzazione dei parametri TensorFlow, lo split dei dati NumPy, lo shuffle dei mini-batch e il campionamento del testo. Le esecuzioni ripetute sono deterministiche nello stesso ambiente software.

Differenziazione automatica

Per ogni mini-batch, TensorFlow registra i calcoli forward e deriva automaticamente la loss rispetto a entrambi i gruppi di parametri addestrabili:

- tf.GradientTape registra embedding lookup, reshape, moltiplicazione di matrici e loss;
- tape.gradient calcola i gradienti per la matrice di embedding e i pesi di output;
- il gradiente sparso degli embedding viene convertito in un tensore denso per questo piccolo vocabolario;
- assign_sub applica aggiornamenti di gradient descent a entrambe le variabili;
- le loss di training e validation vengono misurate dopo ogni epoca.

```
with tf.GradientTape() as tape:
batch_loss = calculate_loss(...)
embedding_gradient, output_gradient = tape.gradient(
batch_loss, [embedding_matrix, output_weights]
)
embedding_matrix.assign_sub(learning_rate * embedding_gradient)
output_weights.assign_sub(learning_rate * output_gradient)
```

Configurazione training:

```
Learning rate: 1.0
Dimensione batch: 32
Epoche: 100
Seed: 42
```

Risultato del training

```
Loss training iniziale: 3.295823
Loss training finale: 0.504322
Loss validation iniziale: 3.295813
Loss validation finale: 6.635019
Migliore loss validation: 2.841331
Migliore epoca validation: 14
```

La loss di training diminuisce nettamente durante tutta l'esecuzione. La loss di validation inizialmente scende, raggiunge il minimo all'epoca 14 e poi aumenta in modo sostanziale mentre la loss di training continua a migliorare.

Questo mostra un chiaro overfitting: il modello basato su un contesto più ampio apprende molto bene il piccolo training set, ma diventa meno efficace sugli esempi di validation. Il notebook registra l'epoca migliore per l'analisi, ma usa i parametri finali dell'epoca 100 per la generazione.

Controlli su gradienti e parametri

I test verificano che TensorFlow produca gradienti finiti per entrambi i gruppi di parametri addestrabili, che le forme dei gradienti corrispondano alle variabili e che entrambe le variabili cambino realmente durante il training.

```
embedding_gradient, output_gradient = tape.gradient(
loss, [embedding_matrix, output_weights]
)
assert dense_embedding_gradient.shape == embedding_matrix.shape
assert output_gradient.shape == output_weights.shape
assert not np.allclose(initial_embeddings, trained_embeddings)
assert not np.allclose(initial_output_weights, trained_output_weights)
```

Probabilità apprese

Dopo il training, il modello produce una distribuzione di probabilità per il carattere successivo a partire da un contesto di quattro caratteri, invece che da un solo carattere.
Per il contesto mode, le probabilità più alte sono:

```
l: 0.8659   spazio: 0.0584   r: 0.0461
c: 0.0266   s: 0.0024        i: 0.0005
```

Generazione neurale del testo

TensorFlow calcola le probabilità dal contesto corrente di quattro caratteri, mentre un generatore NumPy locale campiona il carattere successivo. Il nuovo carattere viene aggiunto, il carattere più vecchio del contesto viene rimosso e la finestra avanza di una posizione.

```
mode -> l
odel -> ?
...
```

Cinque campioni dal contesto addestrato mode sono tutti l con seed 42. La dimostrazione genera poi 300 caratteri dai parametri finali e lo stesso seed riproduce lo stesso testo.

Controlli inclusi

Le asserzioni del notebook verificano la costruzione delle finestre di contesto, i tipi di tensori e variabili, le forme dei parametri, gli split dei dati, loss finite, probabilità normalizzate, gradienti finiti, modifiche dei parametri, retraining deterministico, generazione riproducibile, lunghezza di output richiesta e contesto iniziale.
Un'esecuzione completa dall'inizio alla fine termina con il messaggio All checks passed.

Cosa insegna la Versione 5

La progressione centrale è:

```
contesto -> ID -> embedding -> concatena -> logits
-> loss -> gradienti -> aggiorna -> scorri -> campiona
```

La Versione 5 modifica il modello predittivo per la prima volta da quando è stato introdotto il bigramma neurale. Il focus didattico è ora sulle rappresentazioni apprese dei token e sull'uso di più caratteri precedenti come contesto.

Interpretazione

Gli embedding sostituiscono i vettori one-hot sparsi con rappresentazioni compatte e addestrabili. Un contesto di quattro caratteri fornisce al modello più informazione locale rispetto alla Versione 4 e le diverse posizioni restano distinguibili dopo la concatenazione dei quattro embedding.

Il modello resta semplice: il contesto concatenato viene inviato direttamente a una proiezione lineare di output. La maggiore capacità permette al modello di adattarsi molto più fortemente al corpus di training, rendendo evidente anche l'overfitting osservato.

Limitazioni

- il contesto ha una lunghezza fissa di quattro caratteri;
- il corpus è piccolo ed è incorporato direttamente nel notebook;
- finestre di contesto sovrapposte possono comparire sia nel training sia nella validation a causa dello split casuale;
- il contesto appiattito non ha stato ricorrente, meccanismo di attention o hidden layer non lineare;
- non ci sono early stopping o ripristino dei checkpoint, quindi la generazione usa i parametri dell'epoca 100 anche se la validation è migliore all'epoca 14;
- la validation non è una stima robusta della generalizzazione a un corpus diverso;
- il testo generato cattura pattern locali più forti ma non ha ancora coerenza a lungo termine.

Conclusione

La Versione 5 estende il modello TensorFlow a caratteri con un contesto fisso di quattro caratteri ed embedding appresi. Gli ID dei caratteri selezionano vettori addestrabili, i vettori del contesto vengono concatenati e sia la matrice di embedding sia i pesi di output vengono appresi con differenziazione automatica.

Il contesto fisso e appiattito mantiene l'implementazione facile da ispezionare e prepara il progetto per la Versione 6, dove il sequence modeling potrà elaborare il contesto come una sequenza ordinata.